Mini-Proyecto: Issue Tracker Kanban

Objetivo y plazo

Construir una aplicación completa para gestionar tickets (issues) de equipos mediante un listado, un tablero kanban y comentarios. Se evaluarán frontend, backend, base de datos, integración, pruebas, CI y despliegue como una sola solución funcional.

Plazo: 24 horas. Inicio: martes 20 de octubre de 2026, 5:00 p. m. Entrega: miércoles 21 de octubre de 2026, 5:00 p. m. Todas las horas corresponden a Colombia (UTC-5).

Se permite usar Claude Code u otras herramientas de AI. El candidato es responsable de comprender, verificar y poder explicar el código entregado. Los requisitos describen resultados esperados; las decisiones de implementación deben justificarse en el README.

Requisitos funcionales obligatorios

1. Autenticación

- Registro e inicio de sesión con email y password; sesiones con JWT y expiración, mediante cookie httpOnly o Bearer. Refresh simple o nuevo inicio de sesión al expirar.
- Cada usuario pertenece a una única organización y tiene un rol: admin o member. El registro crea usuarios member dentro de una organización de demostración documentada. El cliente no puede asignarse el rol admin.

2. CRUD de issues

- Campos: title (requerido), description (markdown opcional), status (todo | in_progress | done), priority (low | med | high), assignee_id (nullable), labels (array de strings), created_at y updated_at. Cada issue pertenece a una organización.
- title no puede estar vacío. Valores por defecto: status = todo y priority = med. Solo se puede asignar un issue a un usuario de la misma organización.

3. Listado y filtros

- Paginación en servidor; filtros combinables por status, priority y assignee; búsqueda por texto en title y description. Orden por updated_at descendente por defecto.

4. Comentarios

- Crear, consultar, editar y borrar comentarios de un issue. Mostrar autor y fecha. Solo el autor puede editar o borrar su comentario, incluso si otro usuario es admin.

Las reglas de acceso y el comportamiento del kanban se detallan en la siguiente página y forman parte del alcance obligatorio.

Permisos y experiencia de uso

Organizaciones y roles obligatorios

Los usuarios pueden ver los issues y comentar dentro de su organización. Las siguientes restricciones deben cumplirse tanto en la interfaz como en el servidor:

- **Admin:** puede crear, editar, asignar, reasignar y borrar cualquier issue de su organización.
- **Member:** puede crear issues asignados a sí mismo y editar únicamente los que tenga asignados, incluido su status y priority. No puede reasignar ni borrar issues.
- **Aislamiento:** ningún usuario puede consultar ni modificar issues, comentarios, historial o usuarios de otra organización, aunque conozca sus identificadores o invoque directamente el servidor.
- La interfaz debe reflejar los permisos: ocultar o deshabilitar acciones no autorizadas. La autorización no puede depender únicamente de estos controles visuales.

No se requieren invitaciones, administración de organizaciones, cambios de organización ni una pantalla de gestión de roles. Los usuarios admin se precargan mediante el seed.

Kanban y detalle de issue

- Tres columnas: To do, In progress y Done. Drag & drop para cambiar el status, respetando los permisos del usuario.
- **Actualización optimista:** la tarjeta cambia de columna inmediatamente. Si falla la operación en el servidor, vuelve a su estado anterior y se muestra un mensaje claro. Al recargar, debe verse el estado persistido.
- Detalle en modal o página con título, descripción renderizada en markdown, comentarios e historial. Mantener coherencia entre detalle, listado y kanban después de los cambios.
- Incluir estados de carga, vacío y error en las vistas principales; feedback de éxito o error al guardar. La interfaz debe ser usable en escritorio y móvil.

Historial de actividad obligatorio

- Registrar la creación del issue y cada cambio efectivo de status, priority o assignee. Mostrar autor, fecha, tipo de evento y valores anterior y nuevo cuando corresponda.
- Un cambio y su evento deben persistirse como una sola operación: si falla el registro del historial, el cambio no debe quedar aplicado. Una operación rechazada no debe generar eventos.
- El historial se consulta desde el detalle. No se puede editar ni borrar directamente desde la aplicación. No se exige conservarlo después de borrar el issue.

Stack, pruebas y CI

Tecnologías obligatorias

- **React y TypeScript** para el frontend; React Query o equivalente para consultas y estado del servidor.
- **TanStack Start con TypeScript** para la aplicación y su lógica de servidor. Se admiten funciones o rutas de servidor.
- **PostgreSQL y Drizzle ORM** para persistencia. Incluir esquema y migraciones versionadas que permitan crear la base de datos desde cero.
- Validar datos en cliente y servidor. Manejar errores de forma consistente, distinguiendo autenticación, permisos, validación y errores internos; no exponer información sensible.
- Passwords con bcrypt o argon2; rate limit mínimo en autenticación; CORS cuando corresponda. Mantener secretos fuera del repositorio y proporcionar un archivo de variables de entorno de ejemplo.

Seed reproducible

- Dos organizaciones: Acme y Beta. En cada una, un admin y dos members con credenciales de prueba documentadas.
- Al menos 30 issues en total, distribuidos entre ambas organizaciones y variados en status, priority, responsable y labels; incluir comentarios y actividad de ejemplo.

Pruebas automatizadas obligatorias

Priorizar pruebas de comportamiento sobre una cantidad elevada. Como mínimo, cubrir los siguientes escenarios:

- **Servidor:** rechazar acceso sin sesión; impedir leer o modificar recursos de otra organización; respetar permisos de member y admin; limitar edición de comentarios a su autor.
- **Persistencia:** verificar que cambiar el status guarda el estado y su evento, y que un fallo al registrar el historial no deja el cambio aplicado.
- **Interfaz:** verificar una interacción de filtros o paginación y la reversión visual de un movimiento kanban cuando falla el servidor.

Las pruebas que verifican persistencia deben ejecutarse contra PostgreSQL de prueba, aislado de los datos del despliegue. Documentar cómo ejecutarlas.

Integración continua obligatoria

- Configurar un pipeline real en el repositorio, ejecutado automáticamente en cada push o pull/merge request, que verifique lint, tipos, pruebas automatizadas y build.
- El pipeline debe proporcionar la base de datos de prueba y aplicar las migraciones necesarias. Incluir su configuración y un enlace a una ejecución exitosa del commit entregado.

Un script local sin ejecución automática en el repositorio no cumple el requisito de CI. El despliegue automático es opcional.

Despliegue y entregables

Despliegue obligatorio

Investigar y elegir servicios gratuitos para desplegar toda la solución en la nube. La aplicación debe estar disponible desde una URL accesible por internet y permitir su evaluación sin ejecutarla localmente.

- Frontend y backend deben ejecutarse en la nube; pueden alojarse juntos como aplicación TanStack Start. PostgreSQL también debe estar alojado en la nube y conectado al backend desplegado.
- Aplicar migraciones y seed en el entorno publicado. Los datos deben persistir entre sesiones y reinicios. Configurar variables y secretos en los servicios elegidos.
- El flujo completo debe funcionar en el entorno desplegado: autenticación, roles, aislamiento, issues, filtros, kanban, comentarios e historial. Documentar posibles tiempos de arranque o limitaciones de los planes gratuitos.

Una entrega que solo funciona en local no cumple el requisito de despliegue. No se exige dominio propio ni contratar servicios de pago.

Entregables obligatorios

1. Repositorio accesible para evaluación

Código fuente, migraciones Drizzle, seed, pruebas y configuración de CI. Incluir scripts o comandos equivalentes para dev, build, lint, typecheck, test, migrate y seed. Se acepta una aplicación integrada o un monorepo.

2. README.md

- URL desplegada y credenciales de ambos roles en ambas organizaciones.
- Instalación y ejecución local; variables de entorno; migraciones, seed y pruebas; pasos para reproducir el despliegue y servicios utilizados.
- Enlace al pipeline exitoso del commit entregado; descripción breve de funciones de servidor o endpoints; decisiones técnicas y trade-offs.
- Uso de AI: herramientas utilizadas, qué se delegó y cómo se verificó el resultado. Declarar plantillas o código base reutilizado; indicar limitaciones y requisitos incompletos con claridad.

3. Aplicación desplegada y funcional

La URL y los accesos deben permitir comprobar la solución correspondiente al código entregado. No se requiere video ni presentación adicional.

Evaluación y límites de alcance

Se evaluarán funcionamiento de extremo a extremo, experiencia de uso, autorización, consistencia de datos, claridad del código, pruebas, CI y despliegue reproducible. No se exige una arquitectura empresarial; se valoran decisiones simples, justificadas y verificables.

Se permiten librerías públicas y herramientas de AI, sin copiar un proyecto completo. La revisión posterior podrá incluir una explicación de decisiones y un cambio pequeño en vivo (20 a 30 minutos, fuera del plazo de construcción).

Extras opcionales: adjuntos con almacenamiento persistente, realtime (WebSocket/SSE), docker-compose para desarrollo y despliegue automático. No sustituyen requisitos obligatorios ni compensan un flujo principal incompleto.